

Android

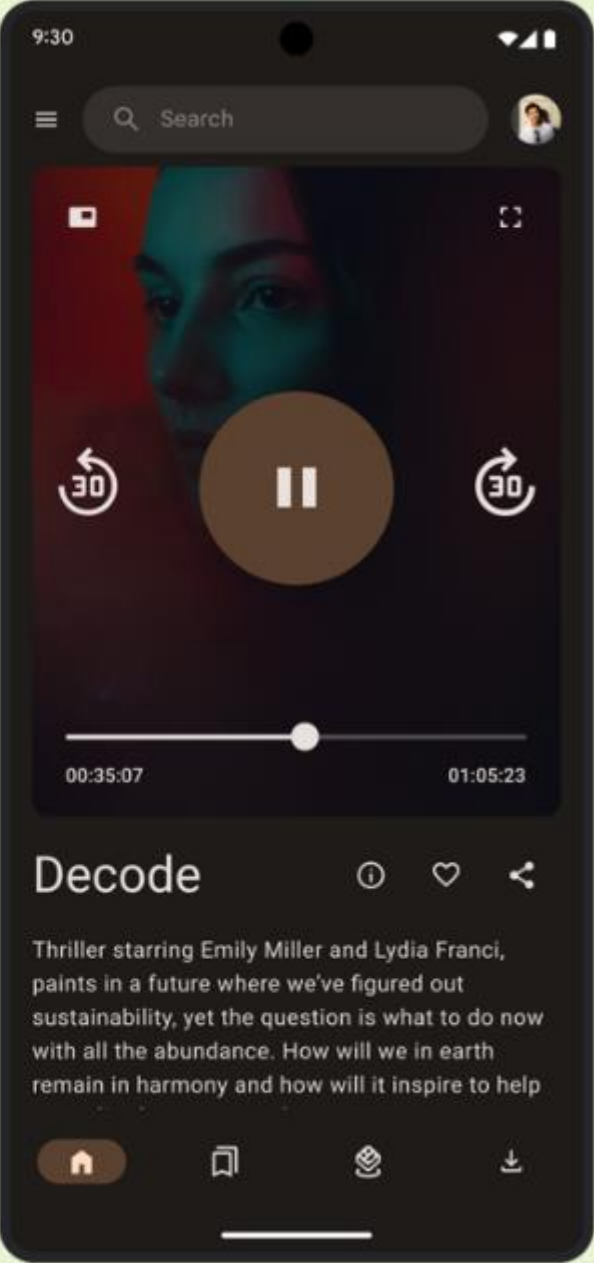
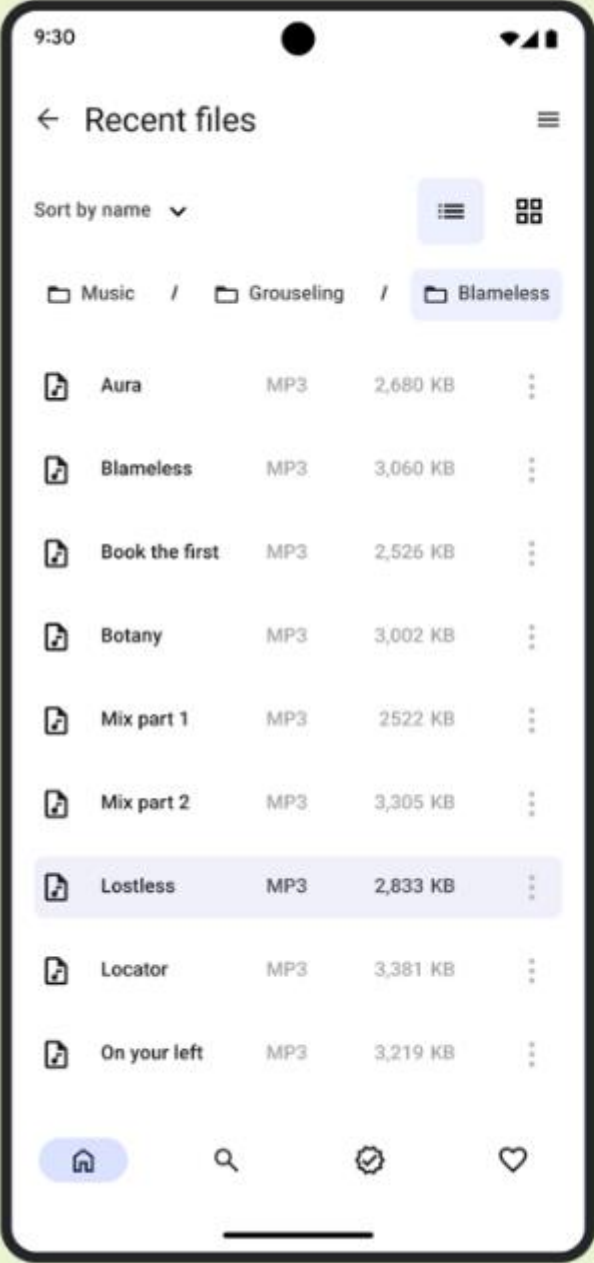
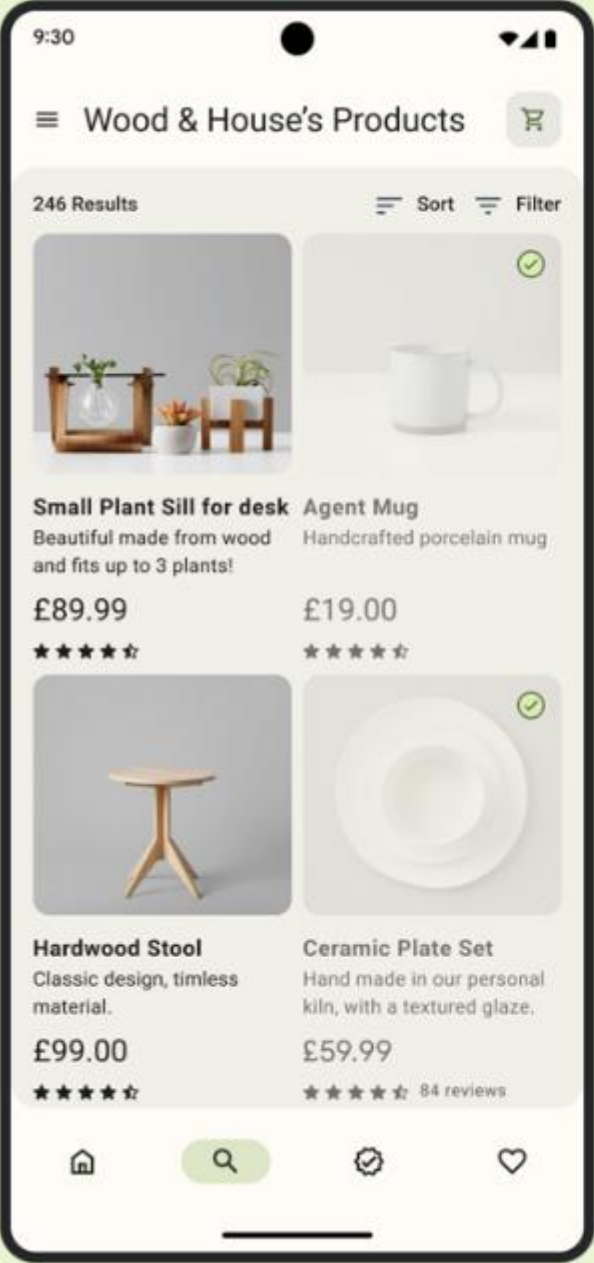
2.

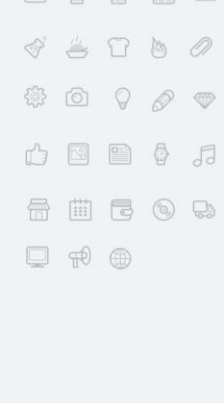
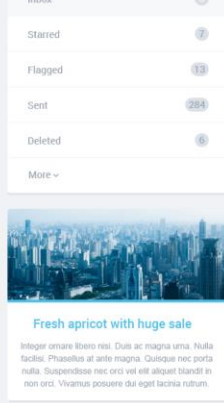
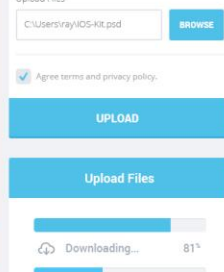
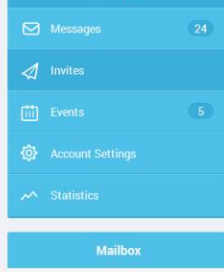
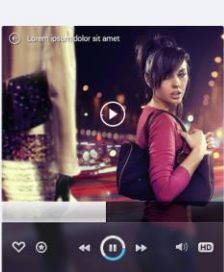
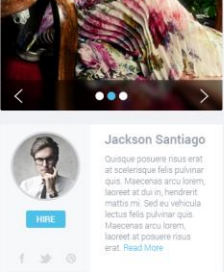
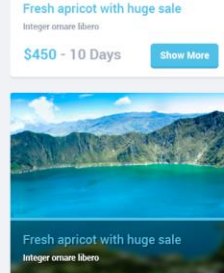
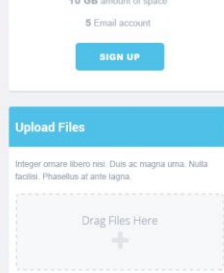
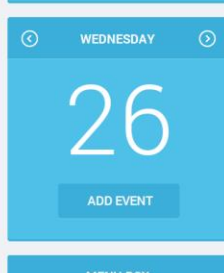
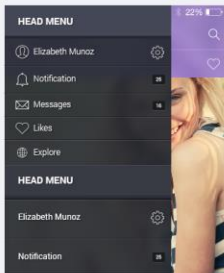
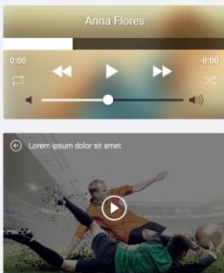
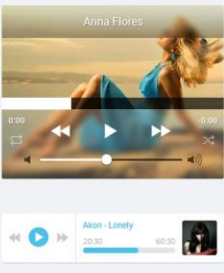
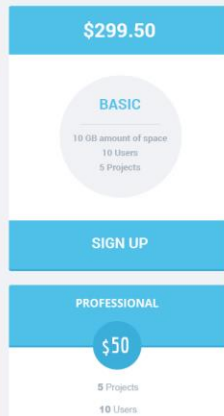
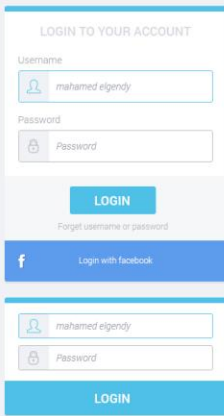
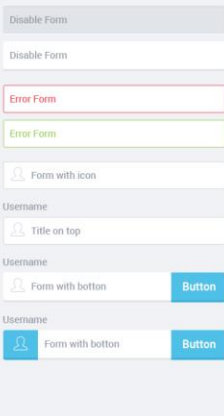
Bolla,
Kálmán

User interface

Importance of the user interface

- First impressions of the user matters
- In Google Play, the user interface is the first thing a user looks at
- All the functionality should be logically organized and clearly visible in order to find what the user need
- Different screen sizes and pixel density make things difficult
- Touch screen quality is important (e.g. multitouch capabilities)





Different resolutions

- From Android 1.6 it is easier to handle different resolutions
- It is necessary to create layouts and images in different sizes
 - the system does the rest for us
- We can write the user interface in an XML file - better separation of the code for the interface and the application logic

Key concepts

- **Screen size**
 - Physical aspect ratio
 - small, normal, large, extra large
- **Screen density (dpi - Dots per inch)**
 - Number of pixels within a given physical area
 - low, medium, high, extra high
- **Orientation**
 - Portrait
 - Landscape

Key concepts

- Resolution (px)
 - Number of screen pixels
- **Density-independent Pixels (dp)**
 - Virtual pixel unit
 - Used in surface design when setting width, height, margin or padding values
 - One dp corresponds to one physical pixel on a 160 dpi (dots per inch) screen
 - $Px = dp * (dpi/160)$

Measurement units

- px: Pixels
- in: Inches (1 Inch = 2.54 cm)
- mm: millimeters
- pt: Points - $1/72$ of an inch based on the physical size of the screen
- **dp: Density-independent Pixels**
- **sp: Scale-independent Pixels (or Scaleable Pixels)**
 - Similar to dp
 - Used to set font size for texts
 - If the user changes the text size in the settings, it will change with the text size

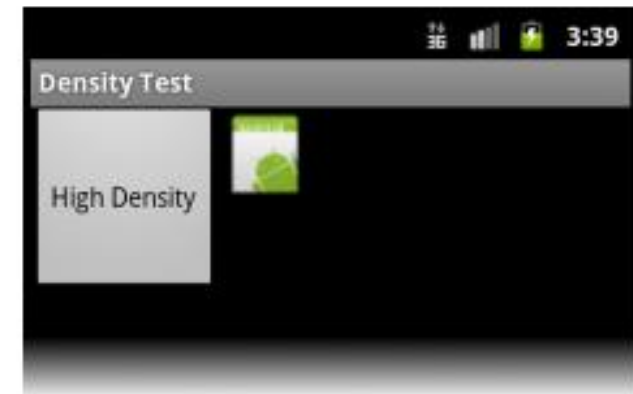
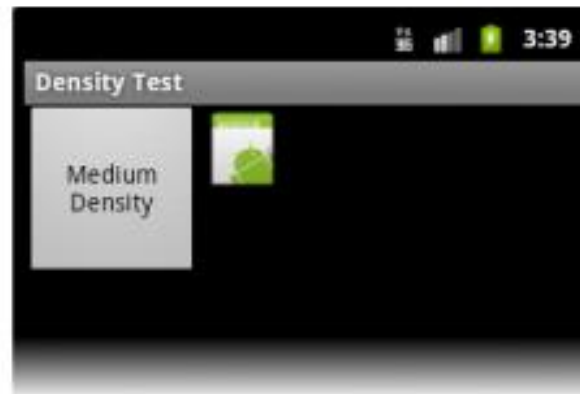
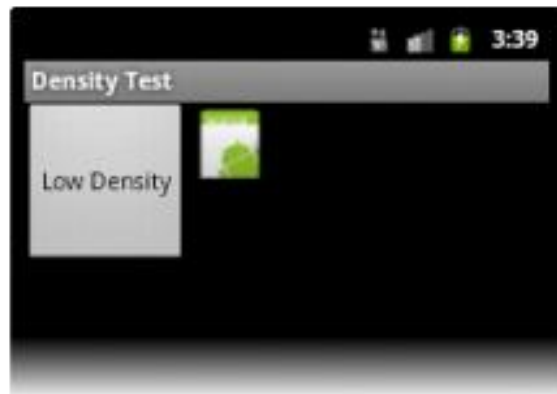
Example

- Density dependent vs independent user interface

px →



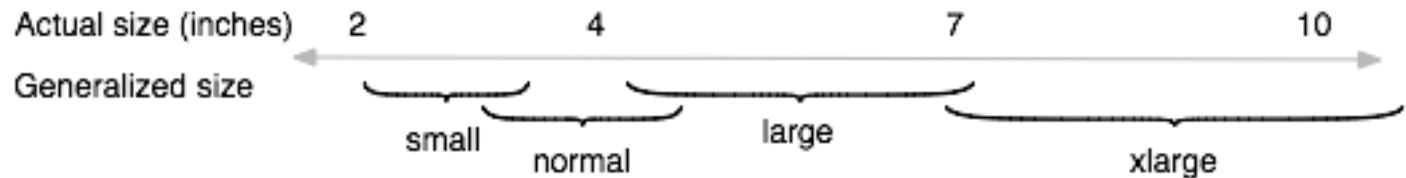
dp →



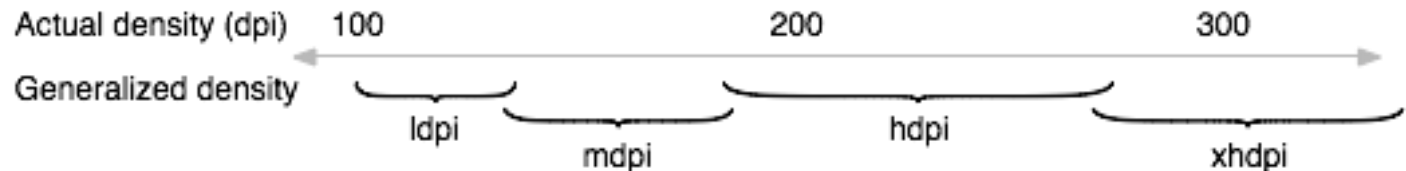
Screen size and density qualifiers

- Qualifiers:
 - low (*ldpi*): ~120dpi
 - medium (*mdpi*): ~160dpi
 - high (*hdpi*): ~240dpi
 - extra-high (*xhdpi*): ~320dpi
 - extra-extra-high (*xxhdpi*): ~480dpi
 - extra-extra-extra-high (*xxxhdpi*): ~640dpi

Screen size



Screen density



Resources

Resources

- Resources: anything that is not a code file, e.g.: images, sounds, interfaces, etc.
- Are stored in a **res** directory, and each resource type has a separate subdirectory
 - E.g.: images are stored in the drawable subdirectory
 - E.g.: audio files are stored in the raw subdirectory
- Use qualifiers: tell the system when to use them
 - E.g.: drawable-xhdpi
 - E.g.: layout-large
- You can combine multiple qualifiers
 - E.g.: drawable-large-hdpi

Resources

- Common use of qualifiers:
 - <resource_name>-<qualifier1>-<qualifier2>-...
- Qualifiers
 - Size: small, normal, large, xlarge
 - Density: ldpi, mdpi, hdpi, xhdpi, nodpi (these are not rescaled by the system), tvdpi
 - Orientation: land, port
 - Aspect: long, notlong
- Example (library for high resolution images):
 - drawable-xhdpi

Resources

- Qualifiers (continued from the previous slide)
 - Width and height constraints:
 - `w<N>dp`: minimum width at which the resource can be used
 - `h<N>dp`: minimum height at which the resource can be used
 - `sw<N>dp`: minimum height and width available on the screen
 - e.g.: `sw600dp`

Resources

- Choosing the right resources:
 - The Android checks if there is a resource that best fits the device parameters
 - If not, it chooses a smaller one (normal instead of large)
 - If there is only a larger resource for the device, an error is returned
- **Manifest file can be used to set if only a certain width or height is supported**

Resources

- Support for all screens and resolutions (manifest file):

```
<supports-screens android:resizeable="true"  
    android:smallScreens="true"  
    android:normalScreens="true"  
    android:largeScreens="true"  
    android:xlargeScreens="true"  
    android:anyDensity="true"/>
```

Resources

- Only landscape/portrait mode display (manifest file):

```
<activity  
    android:name=".MainActivity"  
    android:screenOrientation="landscape"  
    android:theme="@style/AppTheme.NoActionBar">
```

```
<activity  
    android:name=".MainActivity"  
    android:screenOrientation="portrait"  
    android:theme="@style/AppTheme.NoActionBar">
```

Resources

- Location of important resources:
 - User interface: res/layout
 - Text resources: res/values/strings.xml
 - Screen resources: res/drawable
 - Animations: res/anim or res/animation
 - Menus: res/menu

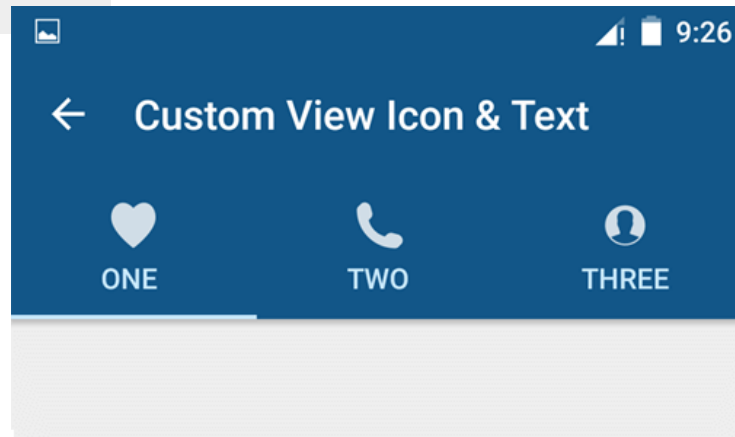
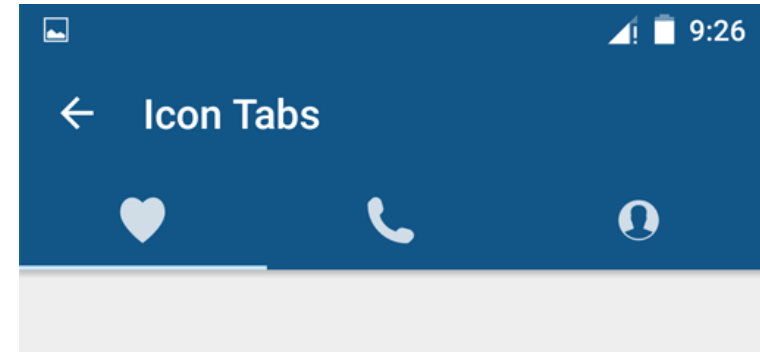
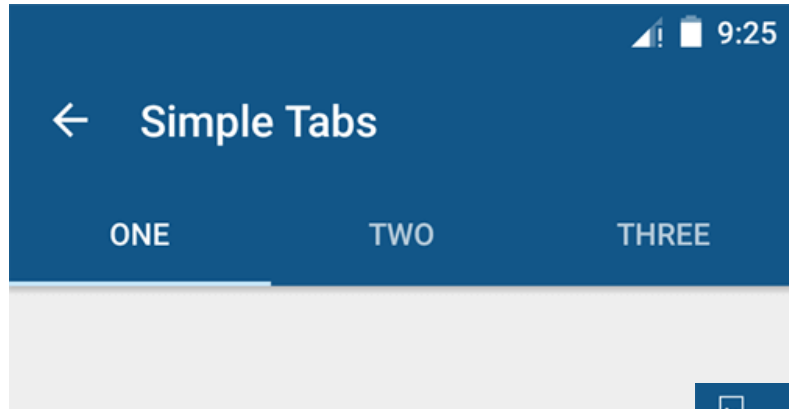
Creating a user
interface

Navigation

- Tabs: your app can be split into 2-3 pages
 - Headers appear at the top
 - Navigate among pages with swipe gesture
- Navigation Drawer: Menu on the left side, which can be opened and closed by the user
 - Menu is used to access the pages and functions of the application
- Action Bar: an element that occupies part of the interface (usually on top) and is visible through the other pages
 - You can place icons, menu items, or use it as a title bar

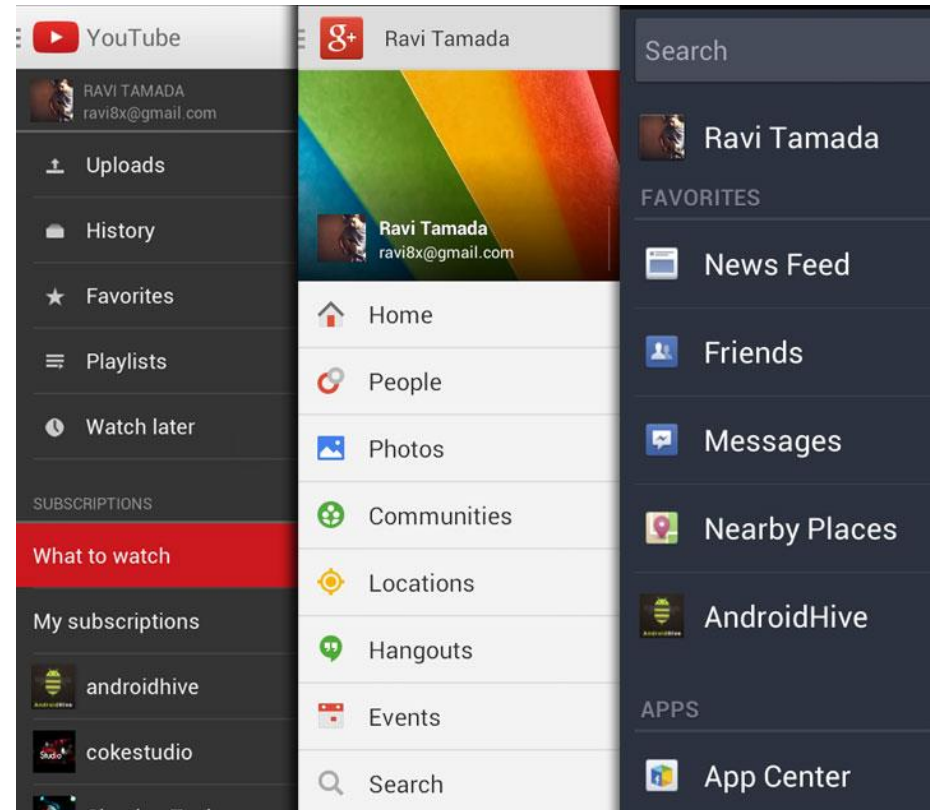
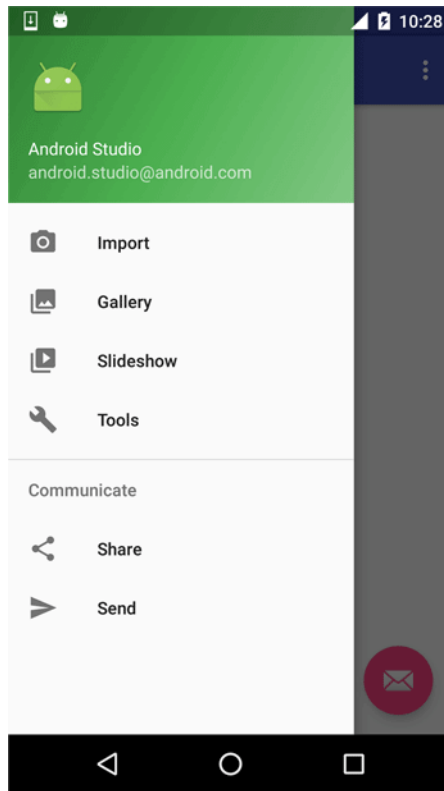
Tabs

- Create swipe views with tabs using ViewPager
 - <https://developer.android.com/guide/navigation/navigation-swipe-view>



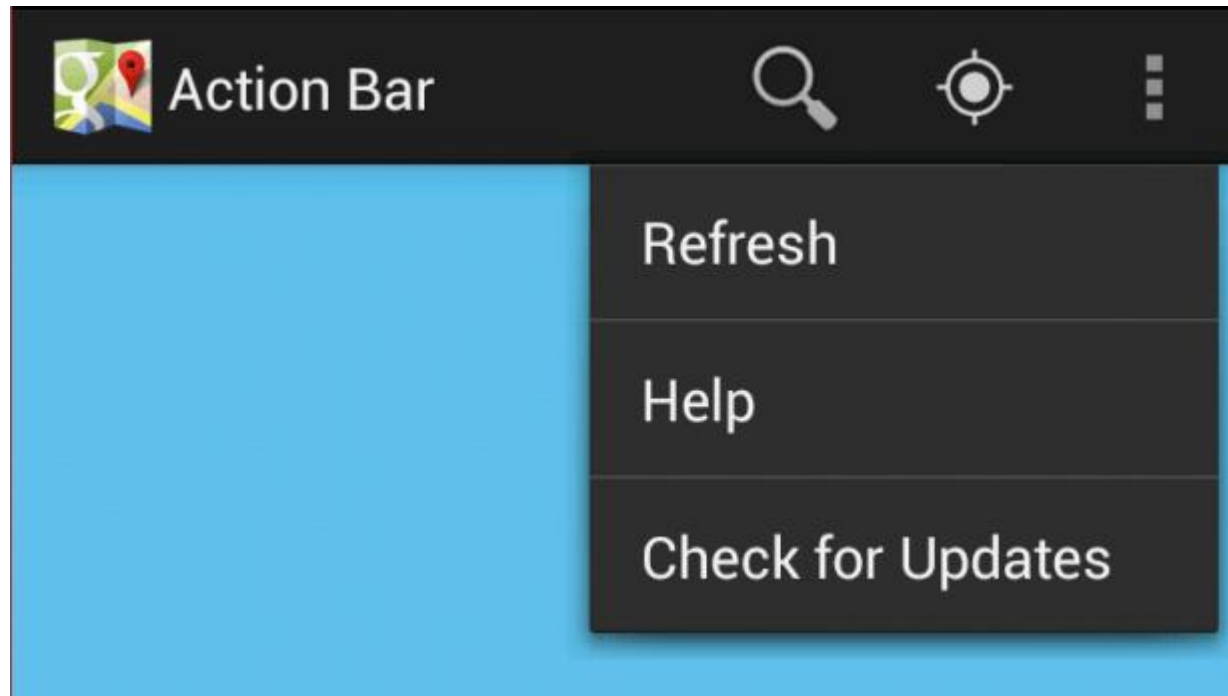
Navigation Drawer

- Update UI components with NavigationUI
 - <https://developer.android.com/guide/navigation/navigation-ui>



Action Bar

- Update UI components with NavigationUI
 - <https://developer.android.com/guide/navigation/navigation-ui>

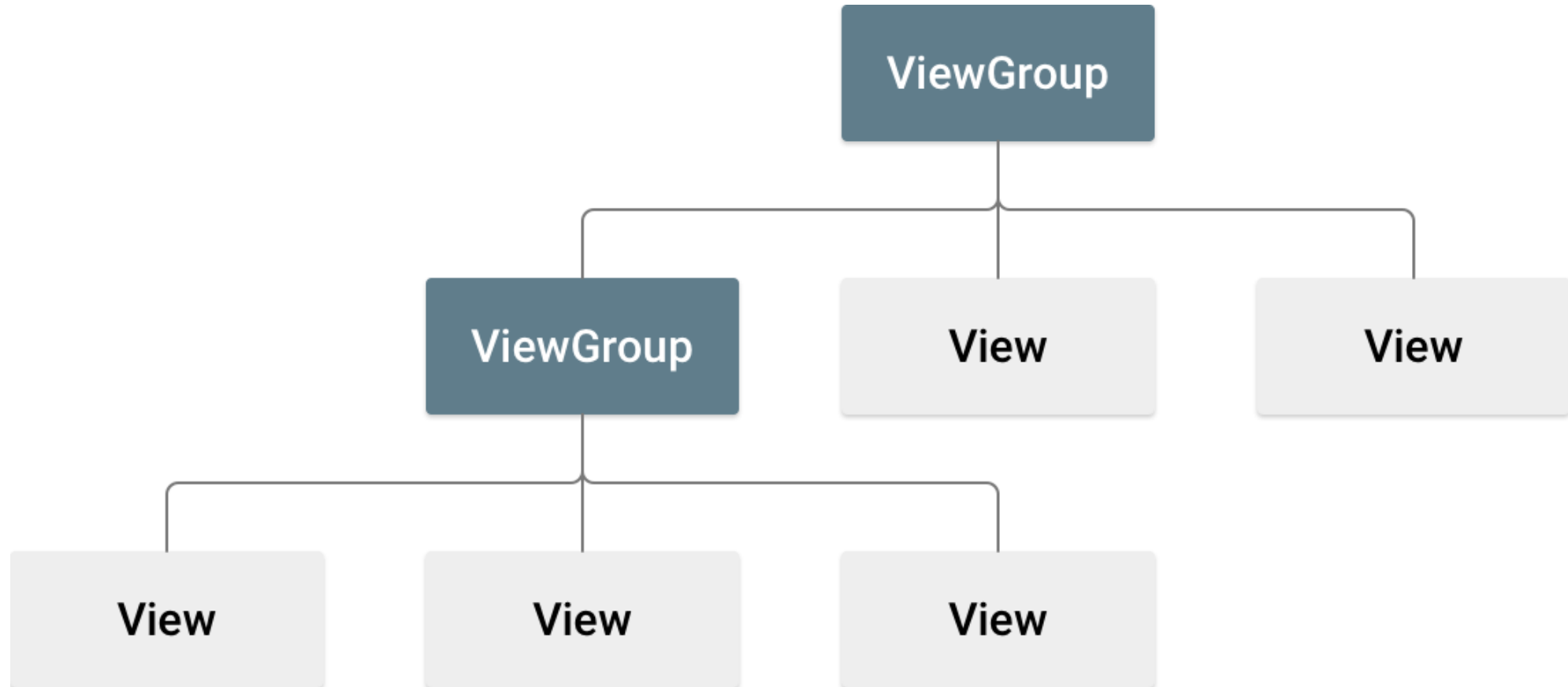


Layout

Layout

- User Interface creation is done with layouts
- Layout defines the layout of the elements
- Each layout is a ViewGroup, which can store elements from a View
- A ViewGroup can contain several ViewGroups
- An Activity usually has one layout: its xml root element is a ViewGroup element

View hierarchy



Linear Layout

- Displays controls below or next to each other
- Multiple Linear Layouts can be nested
 - E.g.: vertical layout has a horizontal layout inside
 - Can contain other types of layout (e.g.: Relative Layout, Frame Layout, etc.)



Linear Layout – orientation

- Controls are positioned vertically, horizontally within the Linear Layout
 - orientation="vertical"
 - orientation="horizontal"

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- ... -->
</LinearLayout>
```

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@android:color/holo_green_light"
    android:orientation="horizontal"
    android:paddingLeft="16dp"
    android:paddingRight="16dp">

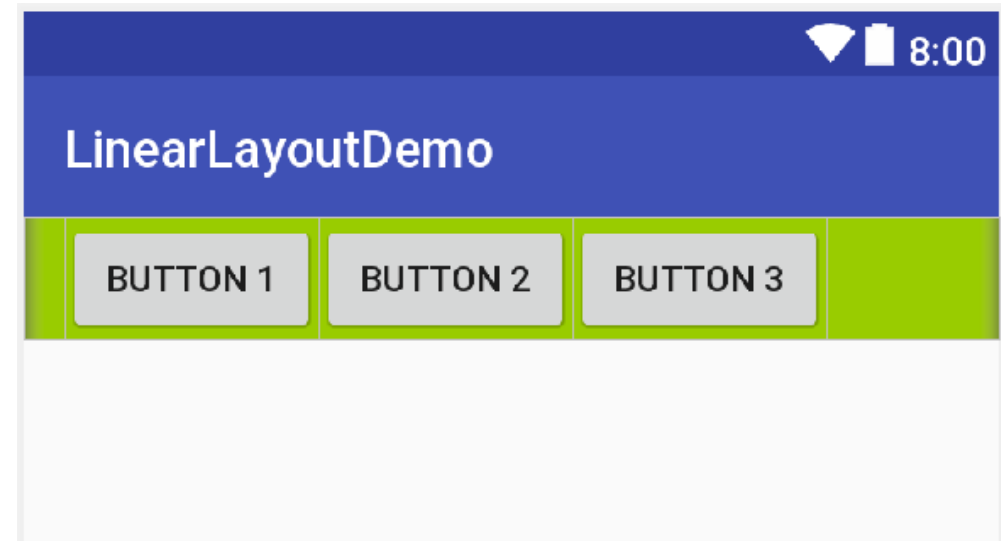
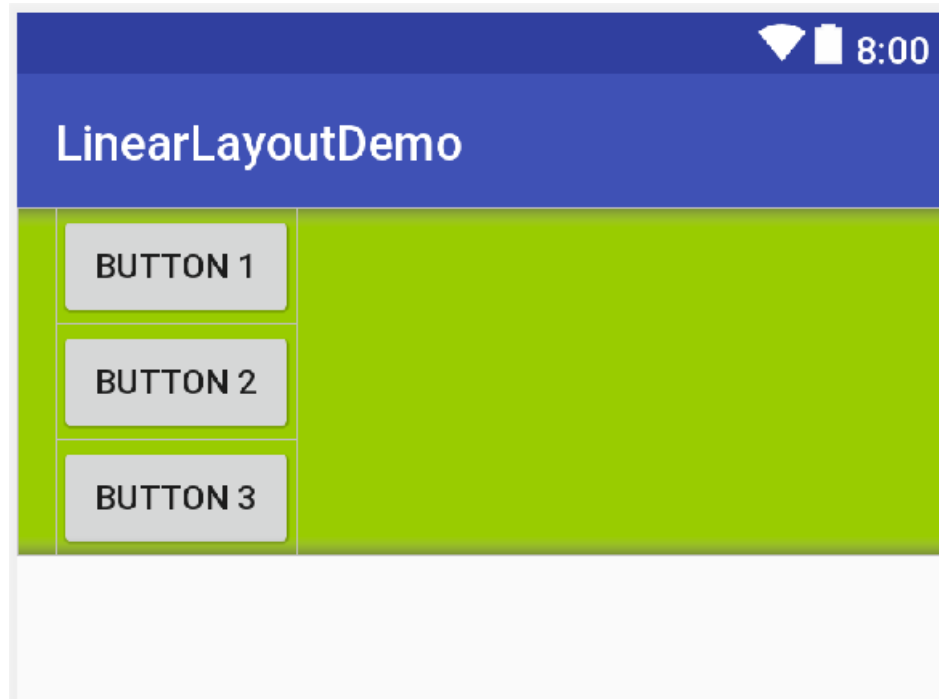
    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:text="Button 1" />

    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:text="Button 2" />

    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:text="Button 3" />

</LinearLayout>
```

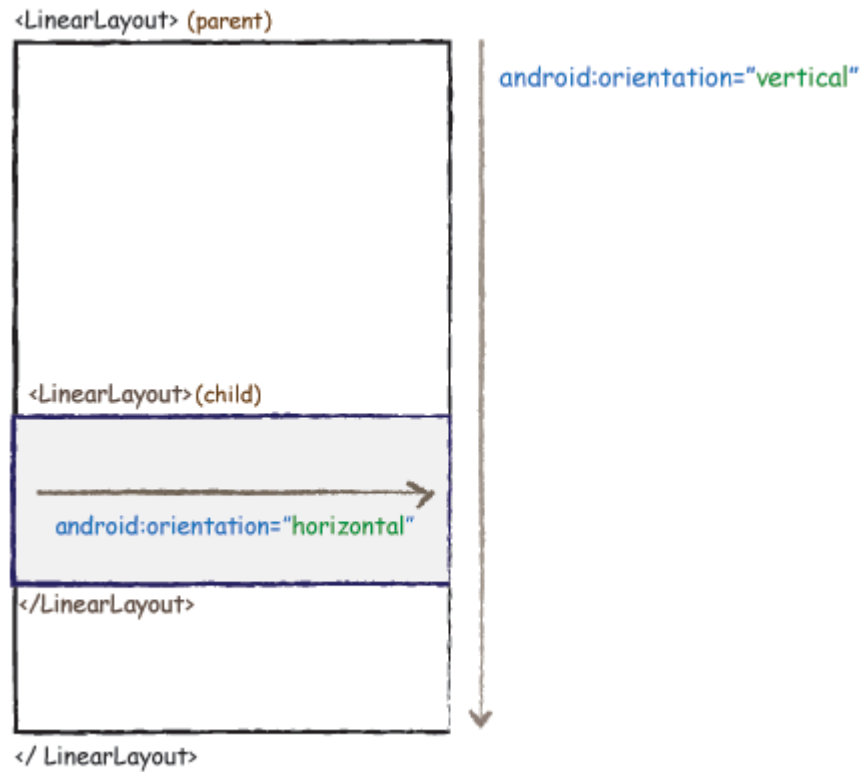
Linear Layout – orientation



Linear Layout – nested

- Multiple Linear Layout (or even other layout controllers, e.g. Relative Layout) can be nested
- Can be used to create complex interfaces
- Disadvantage is that it grows visual tree describing the interface → interface is slower to load

Linear Layout – nested



The screenshot shows a mobile application interface titled "LinearLayoutDemo". The app has a blue header bar with the title and a status bar at the top showing a Wi-Fi icon, a battery icon, and the time 8:00. Below the header, the text "Fill the form" is displayed. Underneath, there is a label "Name:" followed by a text input field containing the word "Name". At the bottom of the form, there are two buttons: "SAVE" and "CANCEL", both highlighted with a pink border.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:textSize="20sp"
        android:text="Fill the form"/>

    <TextView android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Name:"
        android:layout_marginTop="10dp"
        android:layout_marginBottom="10dp"
        android:layout_marginLeft="16dp" />

    <EditText android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Name"
        android:layout_margin="10dp"/>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:background="@color/colorAccent"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <Button android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:layout_gravity="left"
        android:layout_weight="1"
        android:text="Save"/>

    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:layout_gravity="right"
        android:layout_weight="1"
        android:text="Cancel" />

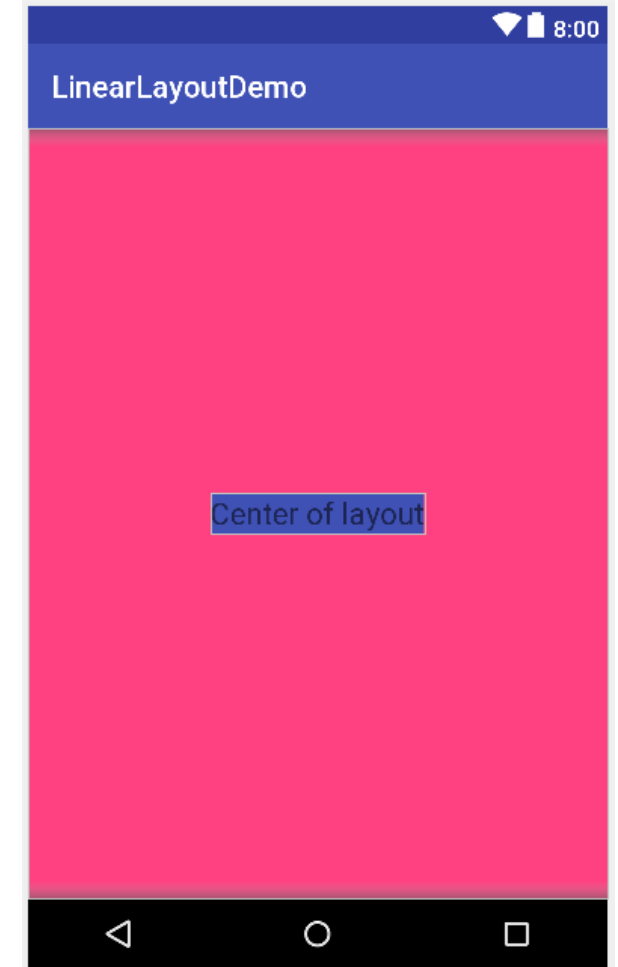
</LinearLayout>
</LinearLayout>
```

Linear Layout – center position

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:background="@color/colorAccent"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@color/colorPrimary"
        android:text="Center of layout"
        android:textSize="20sp" />

</LinearLayout>
```



Linear Layout – weights

- You can also specify weighting in the layout of controls
- To use weights, we need to take care of 3 attributes:
 - Width attribute must be set to 0dp
 - *android:layout_width="0dp"*
 - In the parent element you can specify the sum of the weights (optional)
 - *android:weightSum="5"*
 - The weight of each control is specified (can be an integer or floating point number)
 - *android:layout_weight="1"*

Linear Layout – weights

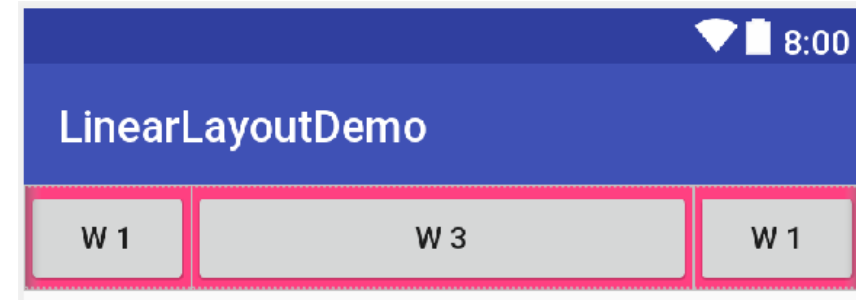
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:background="@color/colorAccent"
    android:weightSum="5">

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="W 1" />

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="3"
        android:text="W 3" />

    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="W 1" />

</LinearLayout>
```



```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:orientation="vertical" >
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/to" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/subject" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="top"
        android:hint="@string/message" />
    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:layout_gravity="right"
        android:text="@string/send" />
</LinearLayout>
```

The screenshot shows a mobile application interface with a blue header bar containing the title "LinearLayoutDemo". Below the header, there are three text input fields stacked vertically, labeled "To", "Subject", and "Message". At the bottom right of the form area, there is a grey button labeled "SEND". The bottom of the screen shows the standard Android navigation bar with back, home, and recent apps icons.

View size attributes

- *android:layout_width* and *android:layout_height* values can be:
 - **match_parent**: get the size of the parent element
 - If you have padding set to so much less
 - ~~**fill_parent**~~: does the same as match_parent
 - Not used from API Level 8
 - **wrap_content**: resize to control content

gravity vs layout_gravity

- We can set the external and internal layout of the controls separately
 - Interior → android:gravity
 - External → android:layout_gravity
- ***android:gravity***: layout of content (e.g. text inside a button) inside the control
- ***android:layout_gravity***: view or layout relative position within the parent element

Relative Layout

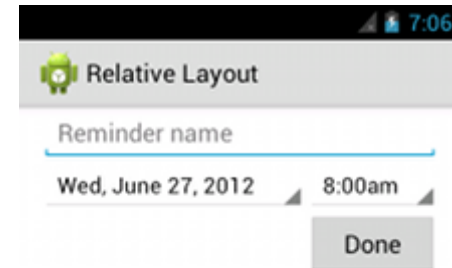
- The relative position of controllers in relation to each other can be given by
 - E.g.: arrange in the middle of the screen
- When the interface is created, the order matters, how the values are set
- For RelativeLayout it is important to give the controls an id



```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp" >
    <EditText
        android:id="@+id/name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/reminder" />
    <Spinner
        android:id="@+id/dates"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentLeft="true"
        android:layout_toLeftOf="@+id/times" />
    <Spinner
        android:id="@+id/times"
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentRight="true" />
    <Button
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/times"
        android:layout_alignParentRight="true"
        android:text="@string/done" />
</RelativeLayout>

```



Constraint Layout

- You can use it to create complex views
- Inherited from Relative Layout, so there is a lot of similarity between the two, but it is much easier to customize
- Relating views to each other works here as well
- Constraints need to be defined
- Guidelines can be used to divide the screen into percentages
- Developer reference:
 - <https://developer.android.com/training/constraint-layout/index.html>

Frame Layout

- Unlike the previous layout controls, it can contain a single element
- Usually are used as a design element

Coordinator Layout

- You should use it in two cases:
 - Implement the top level of an application
 - Creating a list view or a multi-element view
- Example:
 - <http://saulmm.github.io/mastering-coordinator>

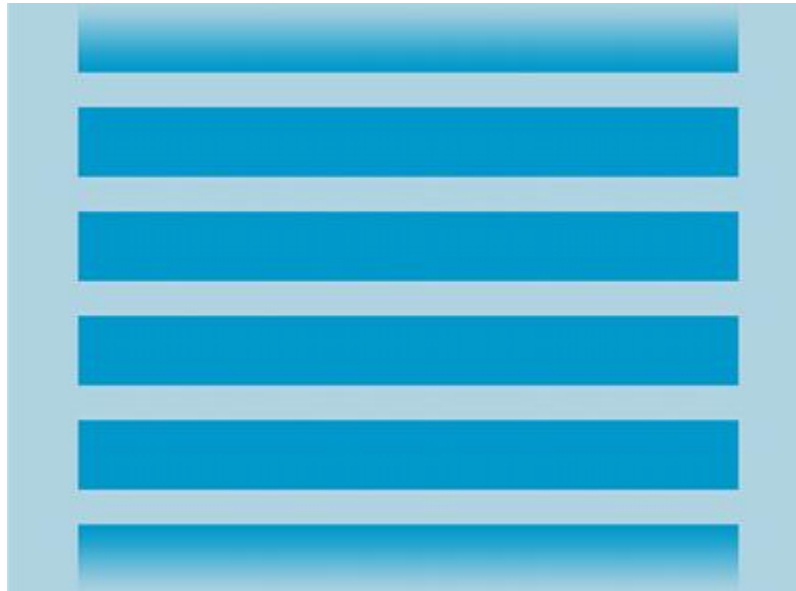
Other layouts

- Web View: can be used to display web content
 - Also used for cross-platform applications, HTML + CSS + JavaScript
- Map View: Google map view

Lists

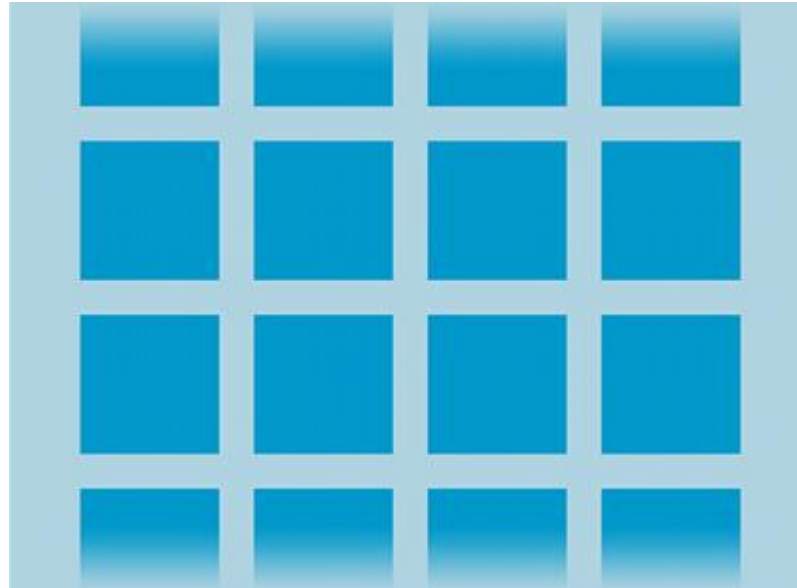
List View

- Displays items as a list, which can be scrolled
- Adapter to specify the items to be displayed
 - Data can come from memory, database, webserver, etc.



Grid View

- Displays the list items in a scrollable grid structure
- You need to create an Adapter to display the list items
- Display can be parameterized in the layout xml (e.g.: number of columns, column width, etc.)



Recycler View

- List View, which has better performance than the plain List View
 - Reuse UI elements
 - Faster than List View
- Developer reference:
 - <https://developer.android.com/guide/topics/ui/layout/recyclerview.html>
 - <https://developer.android.com/training/material/lists-cards.html>

Controls

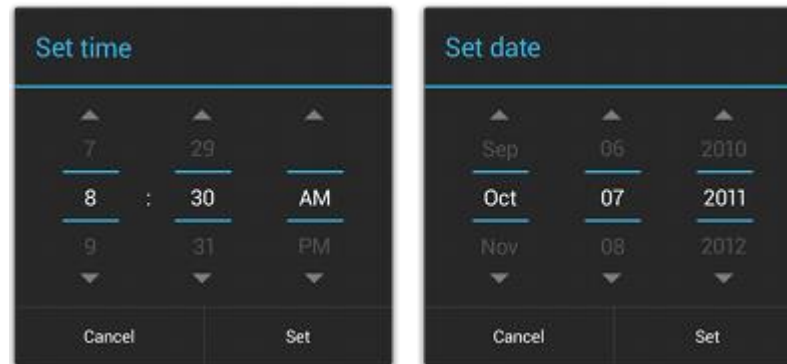
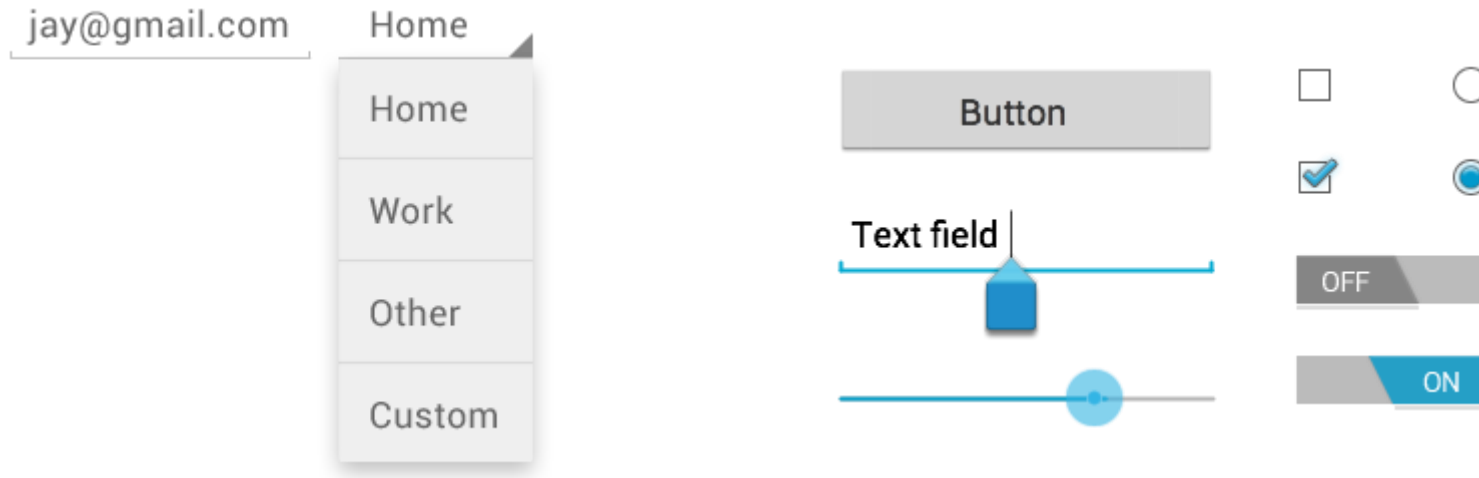
Controls

- Controls can be created programmatically or via XML
General controls:
 - Button, CheckBox, RadioButton, etc.
- Android specific controls:
 - DatePicker, TimePicker, ImageView, etc.

Most commonly used controls

- TextView: text display
- Button
- EditText: input text field
- CheckBox, RadioButton
- ImageView: image display
- Spinner: pick list
- RecyclerView: single or multi-column list
- ToggleButton
- ...

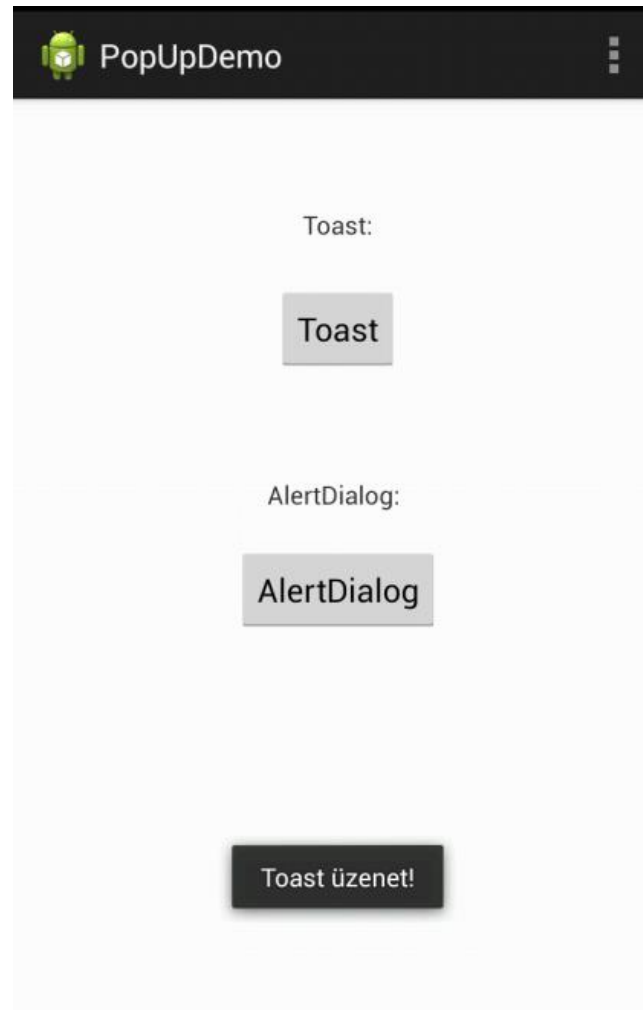
Most commonly used controls



Popup

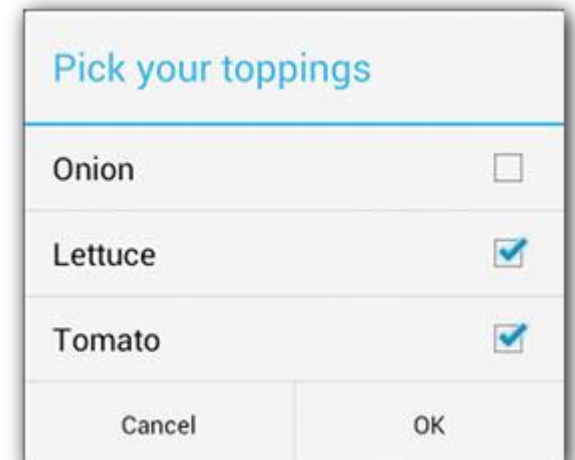
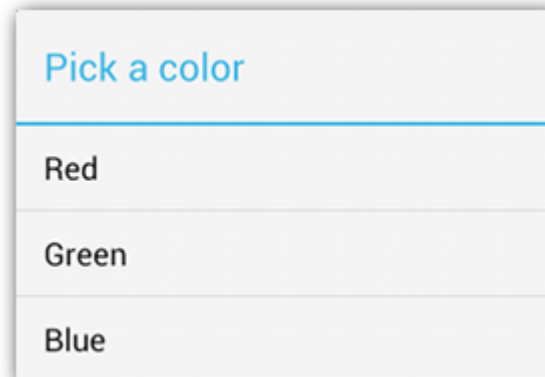
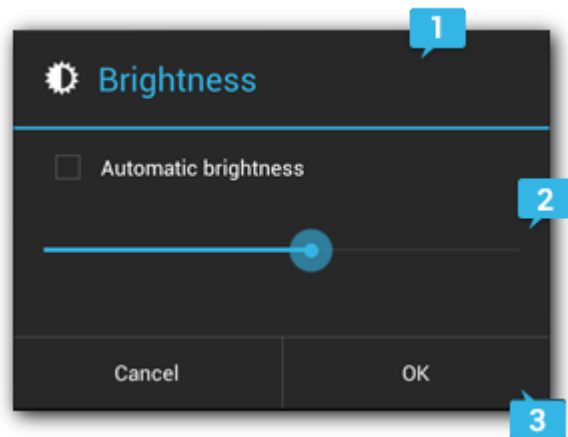
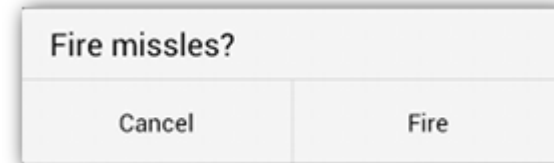
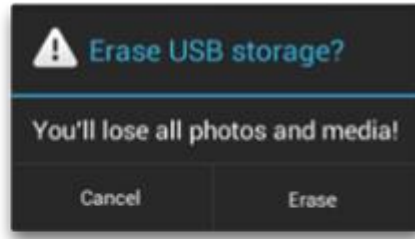
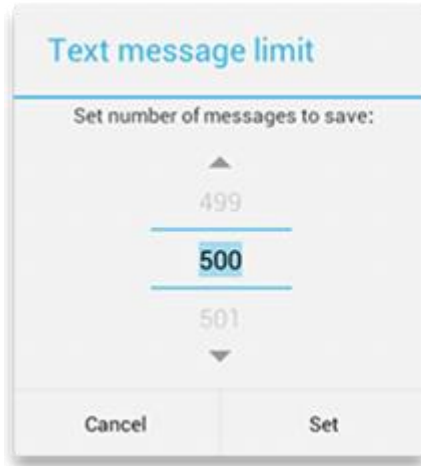
- If you want to create a popup window there are several solutions available:
 - AlertDialog: message window with button
 - Toast: text displayed briefly in a small window
 - SnackBar
 - DialogFragment
 - Popup window displays a whole Activity; the one behind it will be visible but not touchable (you have to set the theme property in manifest)

Toast example



```
Toast.makeText(getApplicationContext(), "Toast üzenet",  
    Toast.LENGTH_SHORT).show();
```

AlertDialog



AlertDialog example

// creating object

```
AlertDialog alertDialog = new AlertDialog.Builder(MainActivity.this).create();
```

// set title text

```
alertDialog.setTitle("Visszaállítás");
```

// set message text

```
alertDialog.setMessage("Biztos vagy benne?");
```

// handling on click event

```
alertDialog.setButton("OK", new DialogInterface.OnClickListener() {  
    public void onClick(DialogInterface dialog, int which) {
```

// further functions by clicking on the button

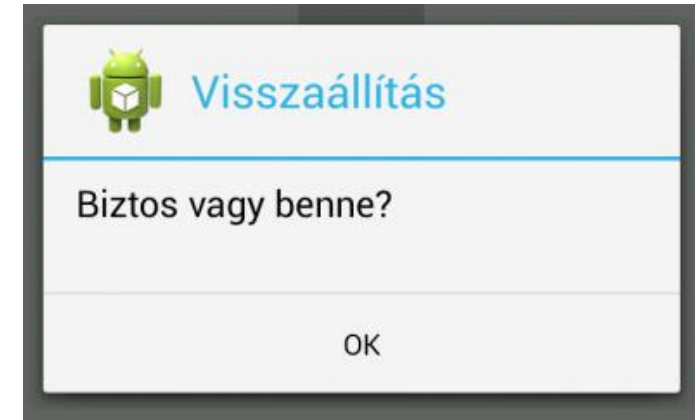
```
    }  
});
```

// icon

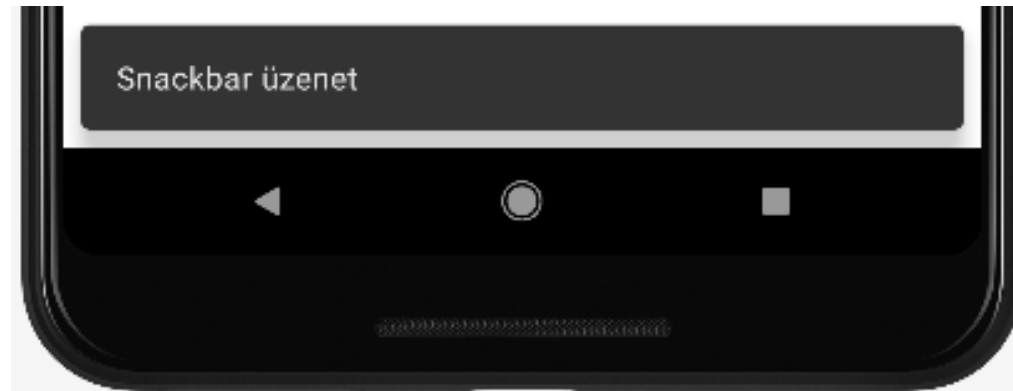
```
alertDialog.setIcon(R.drawable.ic_launcher_foreground);
```

// show dialog window

```
alertDialog.show();
```



SnackBar

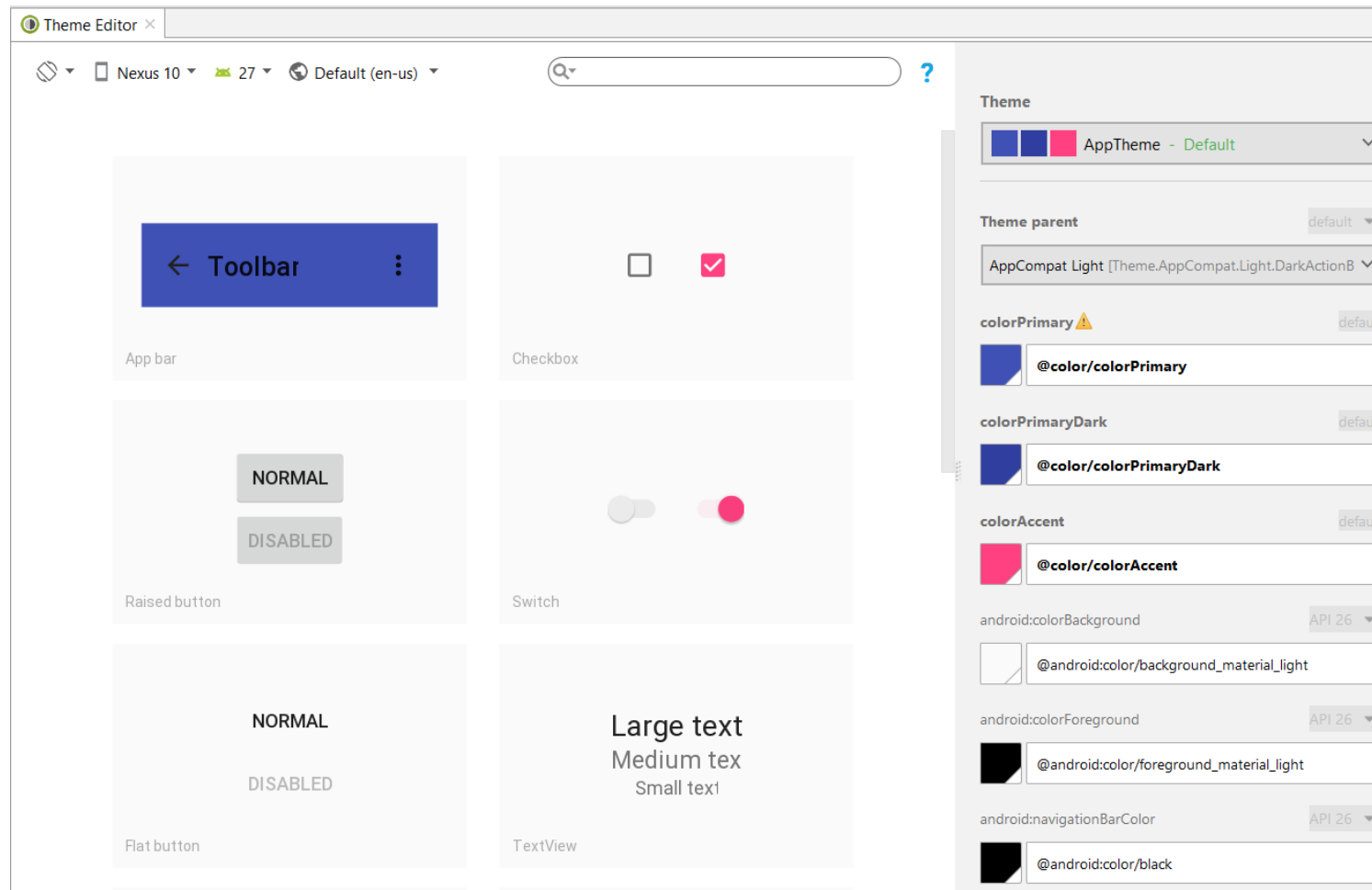


```
View parentLayout = findViewById(R.id.mainLayout);  
SnackBar.make(parentLayout, "SnackBar üzenet",  
    BaseTransientBottomBar.LENGTH_SHORT).show();
```

Design

Android Studio – Theme Editor

- Android Studio → Tools → Theme Editor

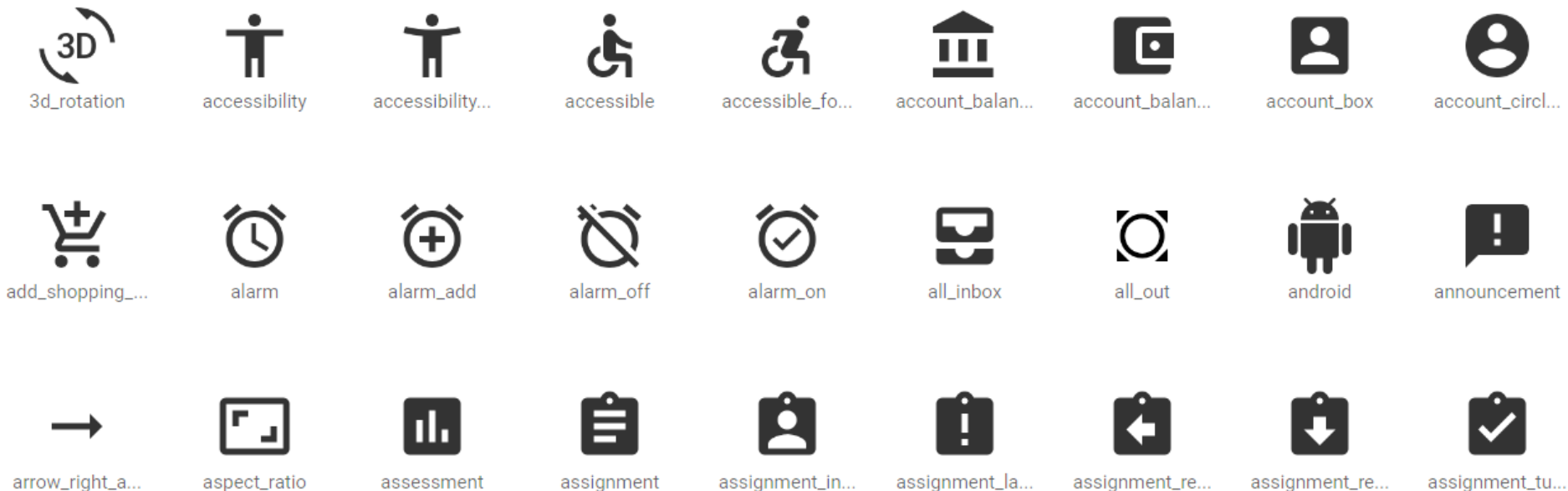


Android Studio – Theme Editor

- You can create, modify an existing theme
- Assign resource IDs to themes
- Visualise changes to individual elements (e.g. how changing colour affects the colour of text)
- Choose colours to get a colour palette
 - Save colour as a resource, so you can use elsewhere
 - @color/.. : color.xml resources for the project
 - @android:color/.. : predefined colors in the android namespace that we can use
- We can inherit from an existing theme → the new theme will be parameterized accordingly, we only need to implement the necessary changes

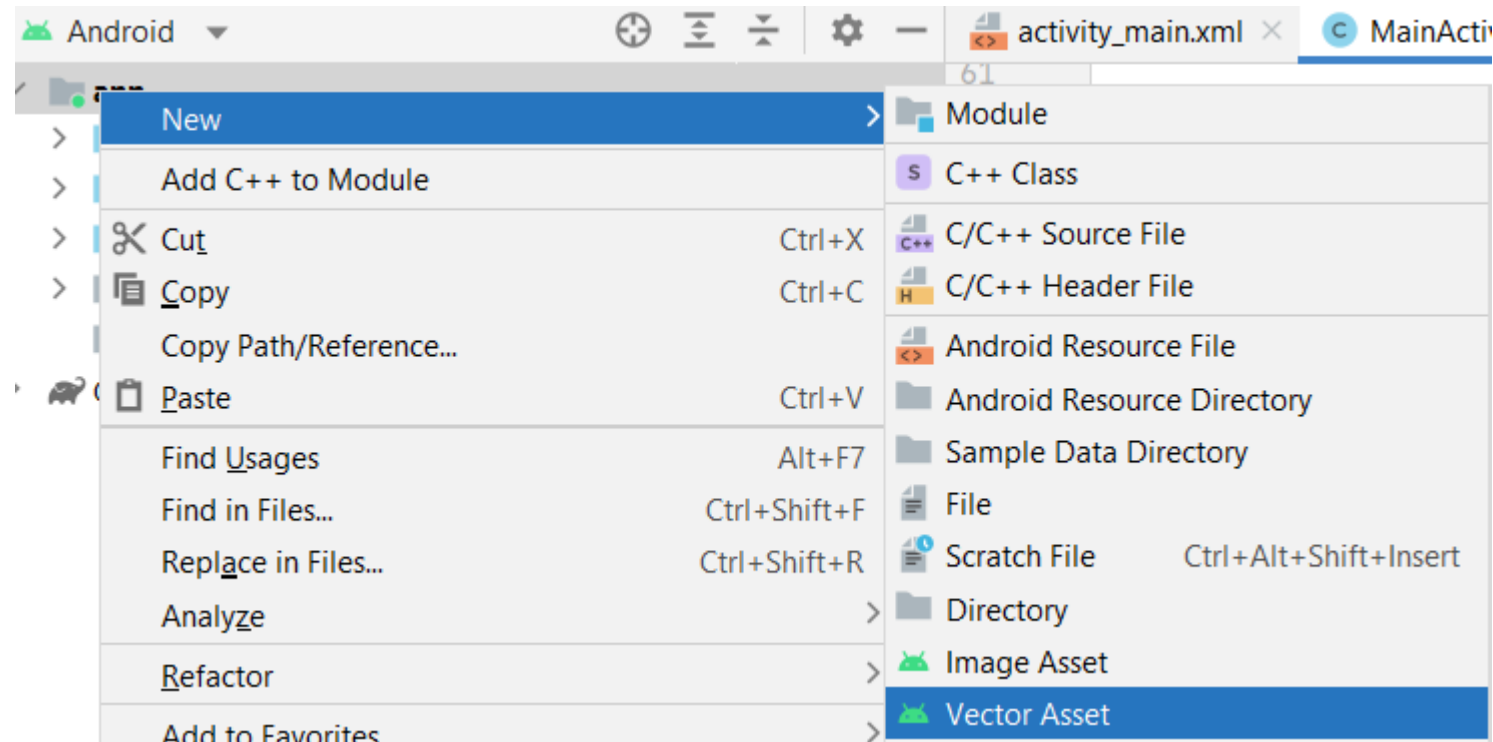
Material icons

- As part of Material Design, we also have a set of icons:
 - <https://material.io/tools/icons/?style=baseline>
- Icons are available in different sizes, colours and shapes

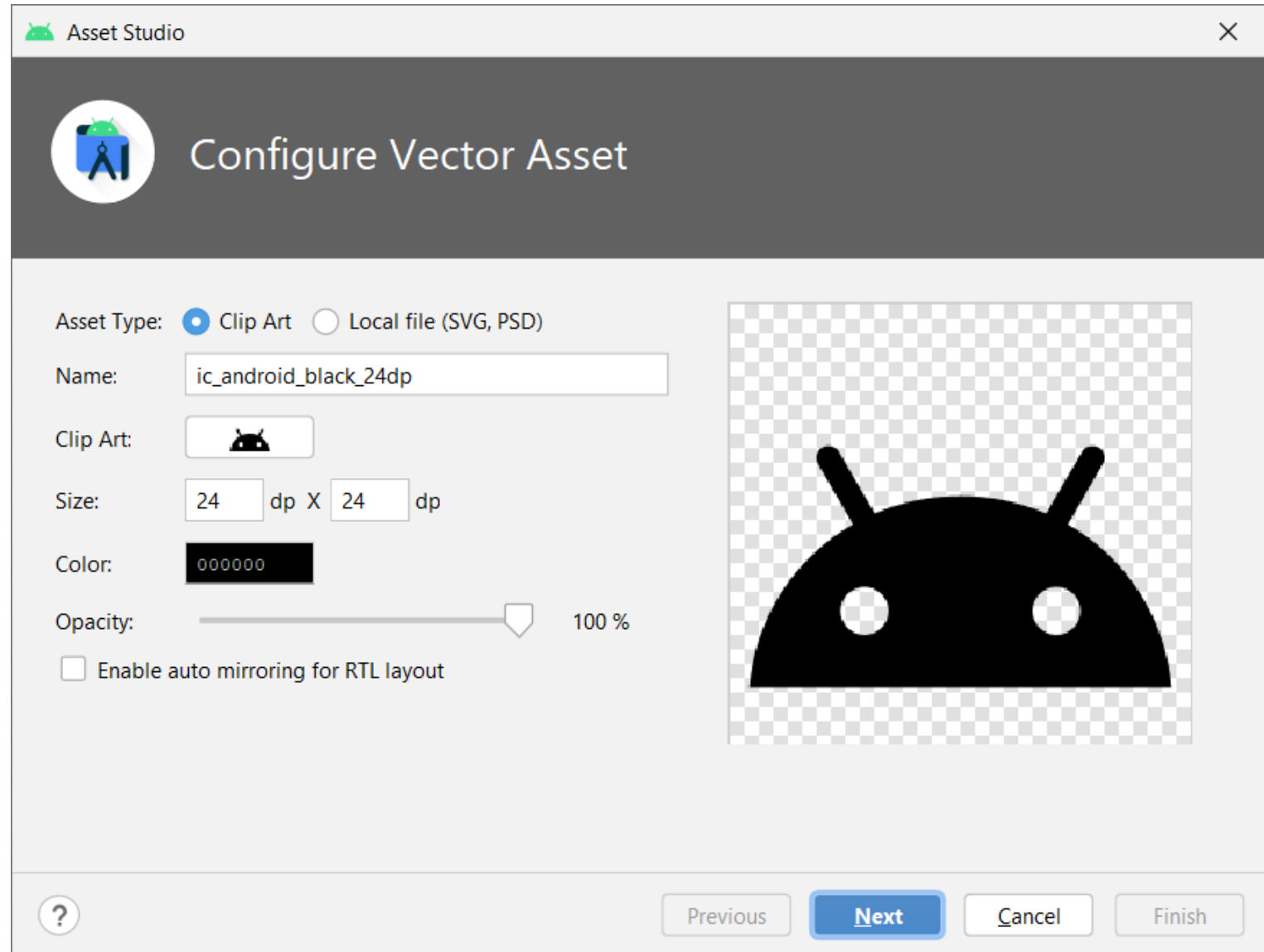


Vector Assets

- Creating vector graphics resources
 - We don't need to produce multiple images of different resolutions from an application icon



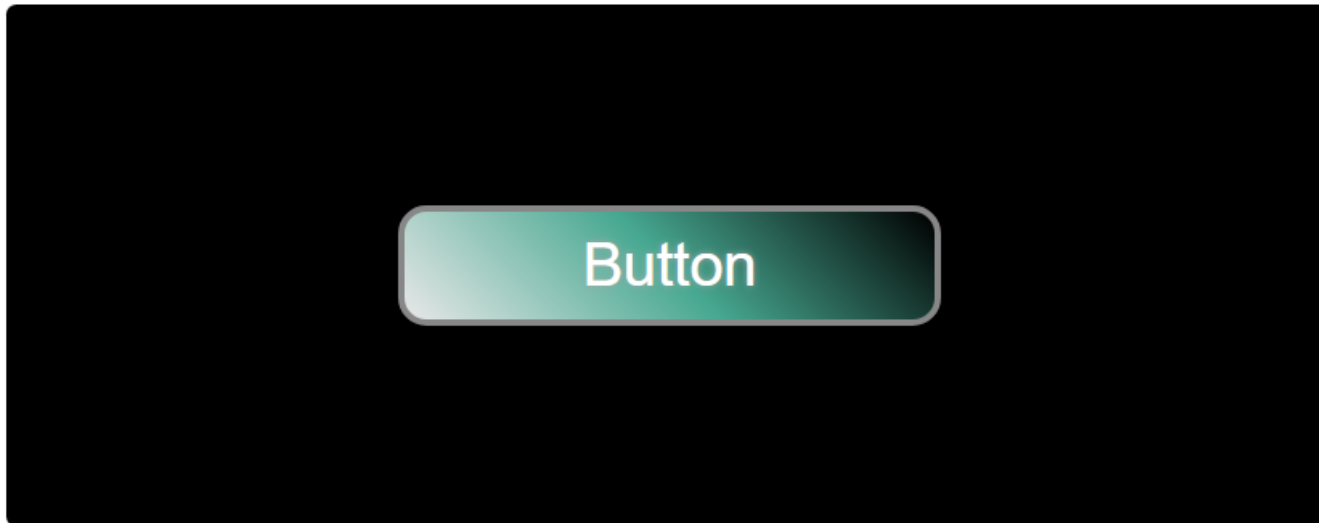
Vector Assets



Button Online Designer

- Android Button Maker
 - <http://angrytools.com/android/button/>

Android **Button** Maker



Button	buttonshape.xml
--------	-----------------

```
<Button
    android:id="@+id/angry_btn"

    android:text="Button"
    android:textColor="#FFFFFF"
    android:textSize="30sp"
```

Button	
android:text	Button
android:textColor	FFFFFF
android:textSize	30
Shadow On ☒	
android:shadowColor	A8A8A8
android:shadowDx	0
android:shadowDy	0
android:shadowRadius	5
Size	
android:width	270
android:height	60
Corners	
android:radius	14
Radius Property ☐	
android:topLeftRadius	0

Resources

- Design app themes with Theme Editor
 - <https://developer.android.com/studio/write/theme-editor>
- Layouts
 - <https://developer.android.com/guide/topics/ui/declaring-layout>
- Designing For Android
 - <http://coding.smashingmagazine.com/2011/06/30/designing-for-android/>

Resources

- Handle configuration changes
 - <https://developer.android.com/guide/topics/resources/runtime-changes>
- LinearLayout
 - <https://developer.android.com/reference/android/widget/LinearLayout>
- Linear Layout
 - <https://developer.android.com/guide/topics/ui/layout/linear>
- Relative Layout
 - <https://developer.android.com/guide/topics/ui/layout/relative>